

AI-Library — Project Review & Learning Record

Date: 5 August 2026
Goal: Follow Microsoft Learn quickstart *Build an AI chat app with .NET* (OpenAI pivot), adapted into an **AI-powered library** web API.
Reference: <https://learn.microsoft.com/en-gb/dotnet/ai/quickstarts/build-chat-app?pivots=openai>
Repo: D:\Repo\AI-Library

What this document is
A learning-oriented review of the current codebase: how it maps to the Microsoft quickstart, what works, what is broken, what should change next, and *why* each recommendation supports learning and the AI Library product idea.

1. What I did (and why)

- 1. **Fetches the Microsoft Learn quickstart** — to compare your project against the official teaching path (IChatClient, chat history, system prompt, streaming).
- 2. **Mapped the full solution structure** — Api / Application / Domain / Infrastructure (Clean Architecture style), plus empty `client`, `docs`, `.github` folders.
- 3. **Read every chat-related source file** — Program, ChatController, ChatCommandHandler, ChatService, ChatHistoryStore, PromptBuilder, DTOs, DI extensions, appsettings, csproj files.
- 4. **Ran `dotnet build`** — to verify compile health and surface real errors/warnings (not only static reading).
- 5. **Wrote this PDF** — so you have a durable learning record of findings, rationale, and a recommended next path aligned with the quickstart + your library idea.

I did not change application code in this pass — review and documentation only, so you can learn from the current state before fixes.

2. Microsoft quickstart vs your project

Quickstart concept	Microsoft sample	Your AI-Library	Status
App host	Console app	ASP.NET Core Web API + MediatR	Adapted (good)
AI abstraction	Microsoft.Extensions.AI + IChatClient	Same packages + IChatClient	Aligned
Model provider	OpenAI / Azure OpenAI	Ollama local (gemma3:4b)	Valid swap (MEAI allows this)

Quickstart concept	Microsoft sample	Your AI-Library	Status
Secrets / config	User secrets for API key + model	appsettings.json for Ollama endpoint/model; UserSecretsId present but unused for AI	Partial
System prompt	Domain-specific hiking guide persona	Generic: "You are a helpful AI assistant."	Needs library persona
Chat history	List<ChatMessage> in memory for one session	ChatHistoryStore with session dictionary	Improved on sample
Streaming	GetStreamingResponseAsync in console loop	Stream through MediatR → API	Broken response shape
Conversation loop	while(true) Console.ReadLine	HTTP POST /api/chat	Correct for API

Learning win: You already went beyond the console sample into layered architecture, DI, MediatR, and multi-session history. That is the right direction for a real AI Library product — keep the quickstart concepts, grow the structure.

3. Current architecture (as implemented)

```
AI-Library.slnx
├─ src/
│   ├── AiLibrary.Api           → HTTP host, ChatController, leftover WeatherForecast
│   ├── AiLibrary.Application   → MediatR command, DTOs, abstractions (IChat*)
│   ├── AiLibrary.Domain        → empty placeholder (Class1 only)
│   └── AiLibrary.Infrastructure → Ollama IChatClient, ChatService, history, prompt
```

- **Dependency direction is correct:** Api → Application + Infrastructure; Infrastructure → Application + Domain; Application → Domain.
- **Domain is unused** — no Book, Member, Loan, Catalog concepts yet.
- **client/** folder is empty — no UI yet.
- **No tests, no README, no CI.**

4. Build result

Command: `dotnet build AI-Library.slnx`

Level	Detail
ERROR	ChatController.cs : ChatRequest has no SessionId . DTO property is ConversationId . Project does not compile .

Level	Detail
WARN	NU1903: Microsoft.OpenApi 2.0.0 high severity vulnerability (via AspNetCore.OpenApi).
WARN	CS8604: possible null Ollama endpoint when building Uri .

5. Issues found (prioritized)

5.1 Critical — must fix to run

#	Issue	Where	Why it matters
C1	Property name mismatch: controller uses <code>request.SessionId</code> ; DTO has <code>ConversationId</code> .	<code>ChatController.cs</code> , <code>ChatRequest.cs</code>	Build fails. Pick one name everywhere (recommend <code>SessionId</code> or <code>ConversationId</code> consistently).
C2	Streaming over HTTP is incorrect: returns <code>Ok(stream)</code> which serializes poorly; clients won't get true SSE/chunked token stream.	<code>ChatController.cs</code>	Quickstart teaching point is streaming UX. API needs <code>Results.Stream</code> / SSE / NDJSON, not JSON-wrapping an <code>IAsyncEnumerable</code> naively.

5.2 High — correctness & product fit

#	Issue	Where	Why it matters
H1	System prompt is generic , not an AI librarian.	<code>PromptBuilder.cs</code>	Quickstart's main lesson: persona + instructions drive useful answers. For AI Library: catalog help, recommendations, citations, ask for genre/age/reading level, etc.
H2	No input validation on message/session (null/empty).	DTO + controller/handler	Nullable disabled in Application; empty messages still hit the model and pollute history.
H3	History list is not thread-safe per session.	<code>ChatHistoryStore</code> + handler	Dictionary is concurrent; <code>List<ChatMessage></code> is not. Parallel requests same session can corrupt history.
H4	History grows forever (no trim / token budget).	handler / store	Long chats exceed context window; cost/latency explode. Quickstart ignores this; real apps must cap or summarize.
H5	Duplicate / dead types: <code>IConversationStore</code> + empty <code>InMemoryConversationStore</code> vs real <code>IChatHistoryStore</code> .	Application + Infrastructure	Confusing for learning; remove or finish one design.

#	Issue	Where	Why it matters
H6	Placeholder Class1.cs in Domain/Application/Infrastructure.	multiple	Noise; delete once real types exist.
H7	WeatherForecast sample leftover in Program.cs.	Program.cs	Unrelated to AI Library; remove to keep mental model clean.

5.3 Medium — quality, security, maintainability

#	Issue	Why
M1	Inconsistent namespaces / missing file-scoped consistency: some types public without matching folder namespace style; controller namespace declared but class not nested consistently.	Harder navigation and discoverability.
M2	Nullable inconsistency: Application has <code><Nullable>disable</Nullable></code> ; other projects enable it.	Hides null bugs; align all projects to enable.
M3	Ollama package is preview 9.7 while <code>Microsoft.Extensions.AI</code> is 10.8.3.	Version skew risk. Prefer matching MEAI.Ollama / current docs package set, or switch to OpenAI package when you follow the OpenAI pivot exactly.
M4	Config nullability: no guard if <code>Ollama:Endpoint</code> / <code>Model</code> missing.	Fails late with unclear errors.
M5	No CORS, auth, rate limiting.	Fine for local learning; needed before any public client.
M6	ChatResponse DTO unused.	Either use for non-streaming endpoint or remove.
M7	OpenAPI vulnerability warning.	Bump package versions when fixing restore.
M8	No README / no run instructions (Ollama must be running, model pulled).	You (and future you) will forget setup steps.
M9	HttpsRedirection after MapControllers order is odd; middleware order should be intentional.	Pipeline clarity.
M10	MediatR returning IEnumerable is awkward (Task.FromResult of async stream).	Works but harder to test/cancel; consider streaming endpoint calling IChatService directly, or a dedicated streaming pipeline.

5.4 Low — polish

- Empty folders: Api Extensions/Middlewares/Configuration, client/, .github/.
- No .editorconfig / Directory.Build.props for shared settings.

- No integration test that hits chat with a fake `IChatClient` .
- `MaxOutputTokens` fixed at 400 — OK for learning; make configurable later.

6. Alignment with “AI-powered library” idea

Today the app is a **generic chat API over Ollama**. It is not yet a library product. To turn the quickstart into your idea, treat chat as the *interface* and the library as the *domain + tools*.

Suggested product vision (learning-friendly MVP)

1. **Librarian chat** — system prompt: recommend books, explain genres, help find titles, ask clarifying questions (age, interests, length).
2. **Catalog data** — Domain entities: Book, Author, Genre; seed a small in-memory or SQLite catalog.
3. **RAG later** — embed book blurbs/reviews; retrieve top-k before answering (next Microsoft Learn AI modules often cover this).
4. **Tools / function calling** — “search catalog”, “get book by ISBN” as AI tools so answers are grounded.
5. **Simple client** — minimal Blazor/React/HTML page that streams tokens (mirrors console loop from quickstart).

Why this order: You solidify quickstart skills (`IChatClient`, history, system prompt, streaming) first, then add domain value without jumping straight into vector DBs.

7. Recommended fix / learning path (ordered)

1. **Unblock build** — unify `SessionId` vs `ConversationId` ; add required validation.
2. **Delete noise** — `WeatherForecast`, `Class1`, unused conversation store stubs (or implement one store only).
3. **Library system prompt** — rewrite `PromptBuilder` like the hiking sample, but for a librarian (questions to ask, how to recommend, tone).
4. **Fix streaming endpoint** — e.g. SSE writing `update.Text` chunks; verify with curl or a tiny HTML page.
5. **Harden history** — lock per session; optional max messages (e.g. keep system + last N).
6. **Config guards** — fail fast if Ollama settings missing; document `ollama pull gemma3:4b` .
7. **Optional OpenAI pivot** — if you want to follow the Learn page literally, add OpenAI package + user secrets (`OpenAIKey` , `ModelName`) behind the same `IChatClient` registration (that’s the point of MEAI abstractions).
8. **Introduce Domain** — Book model + in-memory catalog service injected into prompt or tools.
9. **Add a minimal client** in `client/` to complete the conversational loop experience from the quickstart.
10. **Tests** — handler test with fake `IChatClient` returning fixed stream.

8. Concrete code notes (for when you fix)

8.1 Session id mismatch (C1)

```
// ChatRequest today
public string Message { get; set; }
public string ConversationId { get; set; }

// ChatController today (broken)
new ChatCommand(request.SessionId, request.Message)
```

Either rename DTO → `SessionId`, or pass `request.ConversationId`. Keep command, store, and API contract identical.

8.2 Quickstart-style library system prompt (H1)

```
You are a friendly librarian assistant for an AI-powered library.
Introduce yourself when the user first says hello.
Before recommending books, ask for:
1) preferred genres or topics
2) age group or reading level
3) approximate book length or time available
Suggest up to three titles with a one-sentence why.
If you are unsure a title exists in the catalog, say so.
End by asking if they want more options or a different genre.
```

Why: Same teaching pattern as the hiking prompt — role, required inputs, structured output, polite closing.

8.3 Provider swap (learning MEAI)

Today:

```
services.AddSingleton<IChatClient>(_ =>
    new OllamaChatClient(
        new Uri(configuration["Ollama:Endpoint"]),
        configuration["Ollama:Model"]));
```

OpenAI pivot (conceptually, from the Learn page):

```
new OpenAIClient(key).GetChatClient(model).AsIChatClient();
```

Why: Application/handler code should not care which provider is registered — that is the core Microsoft.Extensions.AI lesson.

9. What is already good

- Using `IChatClient` + streaming API from MEAI (correct abstraction).
- Session-based history with system message seeded once per session.

- Clean Architecture layering early — good habit for growth.
- MediatR command boundary for chat use case.
- Local Ollama — free, private, great for learning without API cost.
- UserSecretsId already on Api project (ready for OpenAI keys later).

10. Out of scope this review

- No production deployment, auth, or multi-tenant design.
- No vector DB / embeddings implementation yet (good next Learn module after chat works).
- No code fixes applied in the repository during this review pass.

11. Suggested immediate next session (you + assistant)

1. Fix build error (session id naming).
2. Remove WeatherForecast + placeholders.
3. Write librarian system prompt.
4. Make `POST /api/chat` stream text properly.
5. Run against Ollama and save 2–3 example dialogues in docs for learning notes.

Bottom line: You adapted the Microsoft chat quickstart into a layered Web API with Ollama — architecturally ahead of the sample, but currently blocked by a DTO/controller mismatch and incomplete streaming/library persona. Fix those, then grow Domain (catalog) on top of the same `IChatClient` patterns. That path maximizes learning from the official docs while building your AI Library idea.

Generated as a project learning record for AI-Library.

Source reference: Microsoft Learn — Quickstart: Build an AI chat app with .NET (OpenAI pivot).

Review method: full source read + `dotnet build` + comparison to official quickstart steps.